Human Verification: `Containers.ComonadConverse`

From Container Comonads Back to Directed Containers (M2b)

Machine-generated from `lean/ComonadConverse.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.ComonadConverse` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

Contents

1 Overview **2**

2 Auxiliary lemmas **2**
- 2.1 Lemma: `Ext.norm_aux` . . . 2
- 2.2 Lemma: `Ext.map_mk` . . . 3
- 2.3 Lemma: `Ext.mk_inj` . . . 3

3 Container comonad data **3**
- 3.1 Structure: `ContainerComonad` . . . 3
- 3.2 Definition: `ContainerComonad.counit` . . . 4
- 3.3 Definition: `ContainerComonad.comult` . . . 4
- 3.4 Structure: `IsComonad` . . . 5

4 Recovering the directed-container operations **5**
- 4.1 Theorem: `ContainerComonad.cShape_eq` . . . 5
- 4.2 Definition: `ContainerComonad.rRoot` . . . 5
- 4.3 Definition: `ContainerComonad.rSub` . . . 6
- 4.4 Definition: `ContainerComonad.rShift` . . . 6
- 4.5 Theorem: `ContainerComonad.comult_norm` . . . 6

5 Recovering D1–D5 **7**
- 5.1 Theorem: `ContainerComonad.rD1` . . . 7
- 5.2 Theorem: `ContainerComonad.rD2` . . . 7
- 5.3 Theorem: `ContainerComonad.rD3` . . . 7
- 5.4 Theorem: `ContainerComonad.rD4` . . . 7
- 5.5 Theorem: `ContainerComonad.rD5` . . . 8

6 The full equivalence **8**
- 6.1 Definition: `ContainerComonad.toDirectedContainer` . . . 8
- 6.2 Theorem: `ContainerComonad.toDirectedContainer_counit` . . . 9
- 6.3 Theorem: `ContainerComonad.toDirectedContainer_comult` . . . 9

7 The other direction **9**
7.1 Definition: `DirectedContainer.toContainerComonad` 9
7.2 Theorem: `DirectedContainer.toContainerComonad_isComonad` 10
7.3 Theorem: `DirectedContainer.toContainerComonad_toDirectedContainer` . . 10

8 Translation conventions **10**

1 Overview

This file proves the **converse** of the Ahman–Chapman–Uustalu equivalence (*When is a container a comonad?*, LMCS 10(3):2014). The forward direction (in `Containers.Directed`) shows that a directed container induces a comonad on its extension. Here we run the construction backwards: from a comonad structure on $\mathbf{Ext}\,C$ we **recover** a directed container and prove that the recovered structure satisfies D1–D5.

Strategy. By the representation theorem (`Containers.Basic`), any natural transformation between container extensions is a container morphism. So the counit and comultiplication of a comonad on $\mathbf{Ext}\,C$ are determined by combinatorial data:

$$\begin{aligned} &\varepsilon\langle s, v\rangle = v(e(s)), && \text{a root choice } e,\\ &\delta\langle s, v\rangle = \langle \mathtt{cShape}(s),\ \lambda p.\langle \mathtt{cSub}(s,p),\ \lambda q.\ v(\mathtt{cVal}(s,p,q))\rangle\rangle, && \text{three components.}\end{aligned}$$

The **linchpin**: the right counit law forces `cShape` = id (`cShape_eq`). Once the outer shape collapses, $(e, \mathtt{cSub}, \mathtt{cVal})$ are exactly a root, sub-shape, and shift, and the three comonad laws become D1–D5 — the same partition as the forward direction, read in reverse.

Everything is **Type**-level, Lean 4 core, no Mathlib.

2 Auxiliary lemmas

2.1 Lemma: `Ext.norm_aux`

Proposition 2.1 (Shape-collapse normalisation)**.** Let C be a container, X a type, $s, t : C.\mathrm{Shape}$, and $h_c : t = s$. Then for any $\mathtt{sub} : C.\mathrm{Pos}(t) \to C.\mathrm{Shape}$ and $\mathtt{val} : (p : C.\mathrm{Pos}(t)) \to C.\mathrm{Pos}(\mathtt{sub}(p)) \to C.\mathrm{Pos}(s)$ and $v : C.\mathrm{Pos}(s) \to X$,

$$\langle t,\ \lambda p.\langle \mathtt{sub}(p),\ \lambda q.\ v(\mathtt{val}(p,q))\rangle\rangle \;=\; \langle s,\ \lambda p.\langle \mathtt{sub}(h_c^{-1}{}_* p),\ \lambda q.\ v(\mathtt{val}(h_c^{-1}{}_* p, q))\rangle\rangle$$

in $\mathbf{Ext}\,C\,(\mathbf{Ext}\,C\,X)$. This absorbs every `cShape`-transport in `comult_norm`.

Lean.

```
theorem Ext.norm_aux {C : Container} {X : Type} {s t : C.Shape} (hc :
    t = s)
    (sub : C.Pos t → C.Shape) (val : (p : C.Pos t) → C.Pos (sub p)
        → C.Pos s)
    (v : C.Pos s → X) :
    (⟨t, fun p ⇒ ⟨sub p, fun q ⇒ v (val p q)⟩⟩ : Ext C (Ext C X))
      = ⟨s, fun p ⇒ ⟨sub (hc.symm ▸ p), fun q ⇒ v (val (hc.symm
          ▸ p) q)⟩⟩
```

Haskell.

```haskell
-- When cShape s = s (propositionally), we can normalise the outer
   shape.
-- In Haskell this is trivial because there are no dependent
   transports;
-- the equality would be definitional.
```

2.2 Lemma: `Ext.map_mk`

Proposition 2.2 (Map on a literal pair)**.** For $f : X \to Y$, shape s, and labelling $g : C.\mathrm{Pos}(s) \to X$,

$$\mathbf{Ext}.\mathtt{map}\, f\, \langle s, g\rangle \;=\; \langle s,\; f \circ g\rangle.$$

This fires only once the argument is in pair form, enabling normalisation to rewrite the inner comultiplication before unfolding **Ext**.`map`.

Lean.

```
theorem Ext.map_mk {C : Container} {X Y : Type} (f : X → Y) (s : C.
   Shape)
    (g : C.Pos s → X) : Ext.map f (⟨s, g⟩ : Ext C X) = ⟨s, fun i ⇒ f
        (g i)⟩
```

Haskell.

```haskell
-- fmap f (Ext s g) = Ext s (f . g)
-- This is just the Functor instance for Ext, applied to a concrete
   pair.
```

2.3 Lemma: `Ext.mk_inj`

Proposition 2.3 (Injectivity of **Ext** pairs)**.** If $\langle s_1, v_1\rangle = \langle s_2, v_2\rangle$ in $\mathbf{Ext}\, C\, X$, then there exists a shape equality $h_s : s_1 = s_2$ such that $\forall q : C.\mathrm{Pos}(s_1),\ v_1(q) = v_2((h_s)_* q)$.

Lean.

```
theorem Ext.mk_inj {C : Container} {X : Type} {s1 s2 : C.Shape}
    {v1 : C.Pos s1 → X} {v2 : C.Pos s2 → X}
    (h : (⟨s1, v1⟩ : Ext C X) = ⟨s2, v2⟩) :
    exists hs : s1 = s2, ∀ q : C.Pos s1, v1 q = v2 (hs cast q)
```

Haskell.

```haskell
-- If two Ext values are equal, their shapes are equal and their
-- labellings agree (up to coercion on positions).
-- In Haskell: if Ext s1 v1 == Ext s2 v2 then s1 == s2 and v1 = v2 .
   coerce.
```

Fidelity note. This is the "destructor" dual to `Ext.ext_eq`: where `ext_eq` *builds* an equality from pointwise data, `mk_inj` *extracts* data from an equality. It is used throughout to read the directed-container laws (D1–D5) out of the comonad-law equalities.

3 Container comonad data

3.1 Structure: `ContainerComonad`

Definition 3.1 (Container comonad)**.** A **container comonad** M extends a container with the combinatorial data guaranteed by the representation theorem:

- $e : (s : S) \to P(s)$ — the counit's *root choice*,
- `cShape` : $S \to S$ — the outer shape of the comultiplication,

- $\texttt{cSub} : (s : S) \to P(\texttt{cShape}(s)) \to S$ — the inner shapes (future sub-shapes),
- $\texttt{cVal} : (s : S) \to (p : P(\texttt{cShape}(s))) \to P(\texttt{cSub}(s,p)) \to P(s)$ — the values (future shift).

The counit and comultiplication are then:

$$\varepsilon\langle s, v\rangle = v(e(s)),$$

$$\delta\langle s, v\rangle = \langle \texttt{cShape}(s),\ \lambda p.\langle \texttt{cSub}(s,p),\ \lambda q.\ v(\texttt{cVal}(s,p,q))\rangle\rangle.$$

Lean.

```
structure ContainerComonad extends Container where
  e : (s : Shape) → Pos s
  cShape : Shape → Shape
  cSub : (s : Shape) → Pos (cShape s) → Shape
  cVal : (s : Shape) → (p : Pos (cShape s)) → Pos (cSub s p) → Pos
     s
```

Haskell.

```
data ContainerComonad s p = ContainerComonad
  { shape   :: s → s          -- outer shape of comult (forced to id)
  , cSub    :: s → p → s      -- inner shapes
  , cVal    :: s → p → p → p  -- future shift
  , e       :: s → p           -- root (counit data)
  }
```

3.2 Definition: `ContainerComonad.counit`

Definition 3.2 (Counit)**.**

$$\varepsilon : \mathbf{Ext}\, M\, X \to X, \qquad \varepsilon\langle s, v\rangle = v(e(s)).$$

Lean.

```
def counit : Ext M.toContainer X → X :=
  fun t ⇒ t.2 (M.e t.1)
```

Haskell.

```
counit :: ContainerComonad s p → Ext s p x → x
counit m (Ext s v) = v (e m s)
```

3.3 Definition: `ContainerComonad.comult`

Definition 3.3 (Comultiplication)**.**

$$\delta : \mathbf{Ext}\, M\, X \to \mathbf{Ext}\, M\, (\mathbf{Ext}\, M\, X),$$

$$\delta\langle s, v\rangle = \langle \texttt{cShape}(s),\ \lambda p.\langle \texttt{cSub}(s,p),\ \lambda q.\ v(\texttt{cVal}(s,p,q))\rangle\rangle.$$

Lean.

```
def comult : Ext M.toContainer X → Ext M.toContainer (Ext M.
    toContainer X) :=
  fun t ⇒ ⟨M.cShape t.1, fun p ⇒ ⟨M.cSub t.1 p, fun q ⇒ t.2 (M.cVal
      t.1 p q)⟩⟩
```

Haskell.

```
comult :: ContainerComonad s p → Ext s p x → Ext s p (Ext s p x)
comult m (Ext s v) = Ext (shape m s)
  (\p → Ext (cSub m s p) (\q → v (cVal m s p q)))
```

3.4 Structure: `IsComonad`

Definition 3.4 (Comonad laws)**.** A container comonad M **is a comonad** if the following three laws hold for all types X:

- **Left counit:** $\varepsilon \circ \delta = \mathrm{id}$,
- **Right counit:** $\mathbf{Ext}.\mathtt{map}(\varepsilon) \circ \delta = \mathrm{id}$,
- **Coassociativity:** $\mathbf{Ext}.\mathtt{map}(\delta) \circ \delta = \delta \circ \delta$.

Lean.

```
structure IsComonad (M : ContainerComonad) : Prop where
  left_counit : ∀ {X : Type},
    M.counit  ∘  M.comult = (id : Ext M.toContainer X → Ext M.
       toContainer X)
  right_counit : ∀ {X : Type},
    Ext.map M.counit  ∘  M.comult = (id : Ext M.toContainer X → Ext M
       .toContainer X)
  coassoc : ∀ {X : Type},
    Ext.map M.comult  ∘  M.comult
      = (M.comult  ∘  M.comult :
          Ext M.toContainer X →
            Ext M.toContainer (Ext M.toContainer (Ext M.toContainer X)
               ))
```

Haskell.

```
-- Comonad laws:
-- left_counit  : extract . comult = id
-- right_counit : fmap extract . comult = id
-- coassoc      : fmap comult . comult = comult . comult
```

4 Recovering the directed-container operations

4.1 Theorem: `ContainerComonad.cShape_eq`

Theorem 4.1 (The linchpin: `cShape` $= \mathrm{id}$)**.** If M is a comonad then $\mathtt{cShape}(s) = s$ for every shape s.

Proof sketch. Probe the right counit law $\mathbf{Ext}.\mathtt{map}(\varepsilon) \circ \delta = \mathrm{id}$ at the generic element $\langle s, \mathrm{id}\rangle : \mathbf{Ext}\, M\, (P(s))$ and read off the shape component.

Lean.

```
theorem cShape_eq (h : M.IsComonad) (s : M.Shape) : M.cShape s = s
```

Haskell.

```
-- The right counit law forces the outer shape map to be the identity.
-- Once cShape = id, (e, cSub, cVal) are exactly (root, sub, shift).
```

Fidelity note. This is the key step of the converse construction. Every dependent cast in the rest of the file is a transport along `cShape_eq`. The Lean proof uses `congrArg` Σ`.fst` to project the shape component from the equality `hr`.

4.2 Definition: `ContainerComonad.rRoot`

Definition 4.2 (Recovered root)**.**

$$o^{\mathrm{rec}}(s) \;:=\; e(s) \;\in\; P(s).$$

Lean.
```
def rRoot (_h : M.IsComonad) (s : M.Shape) : M.Pos s := M.e s
```

Haskell.
```
rRoot :: ContainerComonad s p → IsComonad → s → p
rRoot m _ s = e m s
```

4.3 Definition: `ContainerComonad.rSub`

Definition 4.3 (Recovered sub-shape)**.**

$$s{\downarrow^{\mathrm{rec}}}\, p \;:=\; \mathtt{cSub}\big(s,\ (\mathtt{cShape_eq}(s))_*^{-1} p\big),$$

transporting $p : P(s)$ into the $\mathtt{cShape}(s)$ fibre along $\mathtt{cShape_eq}$.

Lean.
```
def rSub (s : M.Shape) (p : M.Pos s) : M.Shape :=
  M.cSub s ((M.cShape_eq h s).symm cast p)
```

Haskell.
```
rSub :: ContainerComonad s p → IsComonad → s → p → s
rSub m h s p = cSub m s (coerce p)  -- coerce along cShape_eq s
```

4.4 Definition: `ContainerComonad.rShift`

Definition 4.4 (Recovered shift)**.**

$$p \oplus^{\mathrm{rec}} q \;:=\; \mathtt{cVal}\big(s,\ (\mathtt{cShape_eq}(s))_*^{-1} p,\ q\big),$$

transporting p into the $\mathtt{cShape}(s)$ fibre along $\mathtt{cShape_eq}$.

Lean.
```
def rShift (s : M.Shape) (p : M.Pos s) (q : M.Pos (M.rSub h s p)) : M.
    Pos s :=
  M.cVal s ((M.cShape_eq h s).symm cast p) q
```

Haskell.
```
rShift :: ContainerComonad s p → IsComonad → s → p → p → p
rShift m h s p q = cVal m s (coerce p) q  -- coerce p along cShape_eq
    s
```

4.5 Theorem: `ContainerComonad.comult_norm`

Theorem 4.5 (Normalised comultiplication)**.** For any $s : M.\mathrm{Shape}$ and $v : P(s) \to X$,

$$\delta\langle s, v\rangle \;=\; \big\langle s,\ \lambda p.\langle s{\downarrow^{\mathrm{rec}}}\, p,\ \lambda q.\ v(p \oplus^{\mathrm{rec}} q)\big\rangle\big\rangle.$$

The outer shape $\mathtt{cShape}(s)$ has been collapsed to s via $\mathtt{cShape_eq}$. All downstream uses of the comonad laws yield D1–D5 with only the intrinsic transports, exactly mirroring the forward direction.

Lean.
```
theorem comult_norm (s : M.Shape) (v : M.Pos s → X) :
    M.comult (⟨s, v⟩ : Ext M.toContainer X)
      = ⟨s, fun p ⇒ ⟨M.rSub h s p, fun q ⇒ v (M.rShift h s p q)⟩⟩
```

Haskell.
```
-- After normalisation (cShape = id), comult looks exactly like the
-- directed-container comultiplication:
-- comult (Ext s v) = Ext s (\p → Ext (rSub s p) (\q → v (rShift s p
    q)))
```

5 Recovering D1–D5

5.1 Theorem: `ContainerComonad.rD1`

Theorem 5.1 (D1: sub-shape at the root)**.**

$$s\downarrow^{\mathrm{rec}} o^{\mathrm{rec}}(s) \;=\; s.$$

Derived from the left counit law, probing at $\langle s, \mathrm{id}\rangle$.

Lean.
```
theorem rD1 (s : M.Shape) : M.rSub h s (M.rRoot h s) = s
```

Haskell.
```
-- D1:  rSub s (rRoot s) = s
```

5.2 Theorem: `ContainerComonad.rD2`

Theorem 5.2 (D2: shift by the root is the identity)**.** For $q : P(s\downarrow^{\mathrm{rec}} o^{\mathrm{rec}}(s))$,

$$o^{\mathrm{rec}}(s) \oplus^{\mathrm{rec}} q \;=\; (\mathtt{rD1}(s))_{*} q.$$

The value half of the left counit law.

Lean.
```
theorem rD2 (s : M.Shape) (q : M.Pos (M.rSub h s (M.rRoot h s))) :
    M.rShift h s (M.rRoot h s) q = (M.rD1 h s) cast q
```

Haskell.
```
-- D2:  rShift s (rRoot s) q = coerce q  (coerce along rD1 s)
```

5.3 Theorem: `ContainerComonad.rD3`

Theorem 5.3 (D3: shift by the sub-shape's root is the identity)**.** For $p : P(s)$,

$$p \oplus^{\mathrm{rec}} o^{\mathrm{rec}}(s\downarrow^{\mathrm{rec}} p) \;=\; p.$$

The value half of the right counit law.

Lean.
```
theorem rD3 (s : M.Shape) (p : M.Pos s) :
    M.rShift h s p (M.rRoot h (M.rSub h s p)) = p
```

Haskell.
```
-- D3:  rShift s p (rRoot (rSub s p)) = p
```

5.4 Theorem: `ContainerComonad.rD4`

Theorem 5.4 (D4: sub-shapes nest along the shift)**.** For $p : P(s)$ and $q : P(s\downarrow^{\mathrm{rec}} p)$,

$$s\downarrow^{\mathrm{rec}}(p \oplus^{\mathrm{rec}} q) \;=\; (s\downarrow^{\mathrm{rec}} p)\downarrow^{\mathrm{rec}} q.$$

The shape half of coassociativity.

Lean.

```
theorem rD4 (s : M.Shape) (p : M.Pos s) (q : M.Pos (M.rSub h s p)) :
    M.rSub h s (M.rShift h s p q) = M.rSub h (M.rSub h s p) q
```

Haskell.

```
-- D4:  rSub s (rShift s p q) = rSub (rSub s p) q
```

5.5 Theorem: `ContainerComonad.rD5`

Theorem 5.5 (D5: associativity of shift)**.** For $p : P(s)$, $q : P(s\downarrow^{\mathrm{rec}} p)$, $q' : P((s\downarrow^{\mathrm{rec}} p)\downarrow^{\mathrm{rec}} q)$,

$$(p \oplus^{\mathrm{rec}} q) \oplus^{\mathrm{rec}} (\mathtt{rD4}(s,p,q)_*^{-1} q') \;=\; p \oplus^{\mathrm{rec}} (q \oplus^{\mathrm{rec}} q').$$

The value half of coassociativity.

Lean.

```
theorem rD5 (s : M.Shape) (p : M.Pos s) (q : M.Pos (M.rSub h s p))
    (q' : M.Pos (M.rSub h (M.rSub h s p) q)) :
    M.rShift h s (M.rShift h s p q) ((M.rD4 h s p q).symm cast q')
      = M.rShift h s p (M.rShift h (M.rSub h s p) q q')
```

Haskell.

```
-- D5 (shifted associativity):
-- rShift s (rShift s p q) (coerce q') = rShift s p (rShift (rSub s p)
    q q')
-- where coerce is along (rD4 s p q).symm
```

6 The full equivalence

6.1 Definition: `ContainerComonad.toDirectedContainer`

Definition 6.1 (Recovery construction (M2b))**.** Given a container comonad M and a proof $h : \mathtt{IsComonad}(M)$, the **recovered directed container** is

$$(M.\mathrm{toContainer},\ \mathtt{rRoot}_h,\ \mathtt{rSub}_h,\ \mathtt{rShift}_h)$$

with laws `rD1`–`rD5`.

Lean.

```
def toDirectedContainer (h : M.IsComonad) : DirectedContainer where
  toContainer := M.toContainer
  root := M.rRoot h
  sub  := M.rSub h
  shift := M.rShift h
  d1 := M.rD1 h
  d4 := M.rD4 h
  d2 := M.rD2 h
  d3 := M.rD3 h
  d5 := M.rD5 h
```

Haskell.

```
toDirectedContainer :: ContainerComonad s p → IsComonad →
   DirectedContainer s p
toDirectedContainer m h = DirectedContainer
  { root  = rRoot m h
  , sub   = rSub  m h
  , shift = rShift m h
  -- laws D1--D5 proved from the comonad laws
  }
```

6.2 Theorem: `ContainerComonad.toDirectedContainer_counit`

Theorem 6.2 (Counit round-trip)**.** The recovered directed container induces the *same* counit as the comonad we started from, on the nose:

$$\varepsilon_{\texttt{toDirectedContainer}(M,h)} \;=\; \varepsilon_M.$$

This holds definitionally (`rfl`).

Lean.
```
theorem toDirectedContainer_counit (h : M.IsComonad) {X : Type}
    (t : Ext M.toContainer X) :
    (M.toDirectedContainer h).counit (X := X) t = M.counit t
```

Haskell.
```
-- The recovered counit equals the original counit:
-- counit (toDirectedContainer m h) = counit m
```

6.3 Theorem: `ContainerComonad.toDirectedContainer_comult`

Theorem 6.3 (Comultiplication round-trip)**.** The recovered directed container induces the *same* comultiplication as the comonad we started from:

$$\delta_{\texttt{toDirectedContainer}(M,h)}(t) \;=\; \delta_M(t).$$

This is `comult_norm` in reverse, and together with `toDirectedContainer_counit` it says: applying the forward construction (`Containers.Directed`) to the recovered directed container returns the original comonad. This is one half of the equivalence **DirectedContainer** $\simeq$ container comonad.

Lean.
```
theorem toDirectedContainer_comult (h : M.IsComonad) {X : Type}
    (t : Ext M.toContainer X) :
    (M.toDirectedContainer h).comult (X := X) t = M.comult t
```

Haskell.
```
-- The recovered comultiplication equals the original:
-- comult (toDirectedContainer m h) = comult m
```

7 The other direction

7.1 Definition: `DirectedContainer.toContainerComonad`

Definition 7.1 (Packaging a directed container as comonad data)**.** Given a directed container D, its **packaged comonad data** is:

$$M_D = (D.\text{toContainer},\; e := D.\text{root},\; \texttt{cShape} := \text{id},\; \texttt{cSub} := D.\text{sub},\; \texttt{cVal} := D.\text{shift}).$$

Lean.
```
def toContainerComonad (D : DirectedContainer) : ContainerComonad
   where
  toContainer := D.toContainer
  e := D.root
  cShape := id
  cSub := D.sub
  cVal := D.shift
```

Haskell.
```haskell
toContainerComonad :: DirectedContainer s p → ContainerComonad s p
toContainerComonad d = ContainerComonad
  { shape = id, cSub = sub d, cVal = shift d, e = root d }
```

7.2 Theorem: `DirectedContainer.toContainerComonad_isComonad`

Theorem 7.2 (Packaged directed container is a comonad)**.** For any directed container D, the packaged data M_D satisfies the comonad laws (these are exactly the forward laws proved in `Containers.Directed`).

Lean.
```lean
theorem toContainerComonad_isComonad (D : DirectedContainer) :
    D.toContainerComonad.IsComonad where
  left_counit := fun {_} ⇒ D.left_counit
  right_counit := fun {_} ⇒ D.right_counit
  coassoc := fun {_} ⇒ D.coassoc
```

Haskell.
```haskell
-- Directed container laws D1--D5 imply the comonad laws for the
   packaged data.
```

7.3 Theorem: `DirectedContainer.toContainerComonad_toDirectedContainer`

Theorem 7.3 (The other half of the equivalence)**.** For any directed container D,

$$\texttt{toDirectedContainer}(M_D, h_D) \;=\; D$$

definitionally (where h_D is the comonad-law proof above). This holds because `cShape` = id in M_D, so every `cShape_eq`-transport is `rfl` by proof irrelevance.

Together with Theorems **??** and **??**, this establishes the full equivalence:

$$\textbf{DirectedContainer} \;\simeq\; \text{container comonad.}$$

Lean.
```lean
theorem toContainerComonad_toDirectedContainer (D : DirectedContainer)
    :
    D.toContainerComonad.toDirectedContainer
      D.toContainerComonad_isComonad = D
```

Haskell.
```haskell
-- Round-trip: toDirectedContainer (toContainerComonad d) = d
-- This is the definitional equality half of the equivalence.
```

Fidelity note. The Lean proof is `rfl`: the two sides are *definitionally* equal, not just propositionally. This works because `cShape` = id definitionally in M_D, so the recovered `rSub` and `rShift` are definitionally equal to D.sub and D.shift. Proof irrelevance then collapses the law fields. This is a satisfying closure of the equivalence.

8 Translation conventions

1. **Representation theorem.** The whole file rests on the fact (proved in `Containers.Basic`) that natural transformations between container extensions correspond bijectively to container morphisms. This lets us describe the comonad structure combinatorially in terms of $(e, \texttt{cShape}, \texttt{cSub}, \texttt{cVal})$ rather than abstractly as endofunctors.

2. **Generic element probing.** Each directed-container law is *read off* the corresponding comonad law by instantiating at the *generic element* $\langle s, \mathrm{id} \rangle \in \mathbf{Ext}\, M\, (P(s))$ — a Yoneda-style argument.

3. **Dependent transports.** Because $\texttt{cShape}(s) = s$ is only *propositionally* true (not definitionally) in a general `ContainerComonad`, transports along `cShape_eq` appear throughout. The auxiliary `Ext.norm_aux` absorbs all such casts in a single rewrite (`comult_norm`), after which D1–D5 are obtained with only the intrinsic transports from D1 and D4 — matching the forward direction exactly.

4. **Haskell coercion.** In the Haskell renderings, all transports are glossed as `coerce` with a comment indicating the equality they transport along. This is because GHC's `Data.Coerce` provides a safe coercion for representationally equal types, but it cannot capture the full dependent-type content.

5. **Proof bodies omitted.** Only definitions and theorem *statements* are translated. Lean checks the proofs; the human verifier checks that the *statements* match the intended mathematics.